Toward Reusable AI for Edge Offloading: An Empirical Study Under Varying IoT Workloads

Yixin Zhang
School of Science, Engineering and Technology
Penn State Harrisburg, Penn State University
Middletown, PA 17057, United States
yqz6127@psu.edu

Hai Anh Tran
School of Information and Communication Technology
Hanoi University of Science and Technology
Hanoi, Vietnam
anhth@soict.hust.edu.vn

Truong X. Tran
School of Science, Engineering and Technology
Penn State Harrisburg, Penn State University
Middletown, PA 17057, United States
truong.tran@psu.edu

***Abstract*—Deploying reusable AI policies across heterogeneous IoT edge environments requires offloading strategies that remain effective under varying system load, yet this relationship is poorly understood. This paper presents an empirical study of edge offloading under varying load regimes and reinforcement learning (RL) design choices, aimed at identifying lightweight, reusable RL components suitable for edge and IoT deployment. We evaluate MLP-DQN and Attention-DQN variants against a greedy heuristic and trivial baselines across different workload levels. Our results show that under light load, a greedy heuristic nearly matches RL performance ($\sim$99% success rate), but as load increases toward and beyond edge service capacity, simple strategies degrade sharply. The always-offload collapses to $\sim$1.4% success rate under overload, while RL-based methods remain stable above 90%. Within RL configurations, added complexity from attention mechanisms, prioritized experience replay, and history stacking yields marginal and inconsistent gains: latency differences stay within 5%, energy within 0.02 mJ, and success rates within 1%.**

***Index Terms*—Multi-Access Edge Computing, Reusable AI, Deep Reinforcement Learning, IoT System Workload**

I. Introduction

The Internet of Things (IoT) has driven exponential growth in computational demand, pushing resource-limited devices to offload tasks to edge servers. The Multi-Access Edge Computing (MEC) paradigm enables each device to decide whether to execute a task locally or offload it based on system signals such as queue length, channel conditions, and available resources [1]. A central challenge for real-world IoT deployments is designing *reusable* offloading policies, ones that remain effective across varying load conditions without retraining for each deployment context.

The system is typically modeled as a stochastic process with arrival rate λ and service rate μ, framing offloading as a sequential decision-making problem. Heuristic methods are efficient but brittle across conditions [2], [3], while reinforcement learning (RL) offers adaptive policies that capture complex dynamics [4]–[6]. However, increasingly complex RL designs raise a practical question for IoT deployments: does added complexity translate to reusable gains, or does it introduce unnecessary overhead?

We address this through an empirical study across four load regimes ($\mu/\lambda \in \{2.0, 1.6, 1.2, 0.8\}$). Under light load, a greedy heuristic achieves $\sim$99% success rate and matches RL, but under overload it collapses to $\sim$1.4% while RL remains above 90%. Within RL, attention, PER, and history yield gains under 5%, favoring simpler architectures for edge deployment. Our contributions are:

- An empirical analysis of how edge workload affects algorithm selection, with implications for reusable AI in IoT systems.
- A systematic comparison of RL design choices, including architecture, replay strategy, and history, across workload conditions.
- Evidence that lightweight RL architectures match complex ones in near-Markovian MEC settings, supporting practical edge reusability.

The remainder of the paper is organized as follows. Section II reviews related work. Section III presents the system model and MDP formulation. Section IV describes the algorithm's design choices and the experiment results. Section V concludes all previous information.

II. Related Works

Early MEC offloading works rely on traditional optimization approaches and heuristics. Jia et al. [7] propose Ly-MAPPO, a multi-agent Proximal Policy Optimization (PPO) algorithm with long-term constraints for the Internet of Vehicles, while Tan et al. [8] decompose the problem into offloading and resource allocation, solving each via convex optimization. Heuristic methods offer a lighter alternative: HOM [2] selects optimal virtual machines in 5G IoT networks, and HAGP [3] formulates offloading as a MINLP problem solved greedily to optimize CPU frequency and transmission.

More recent work has shifted toward deep reinforcement learning (DRL), formulating offloading as a sequential

decision-making problem over system states such as queue length, channel condition, and energy. OPO combines LSTM with DRL to capture temporal dynamics [9], while Double DQN [10], dueling DQN [11], and federated variants [12] improve training stability. Moon et al. [13] further address suboptimal convergence from local-only information via a hybrid centralized-distributed approach.

Despite these advances, most works focus on model sophistication under fixed system assumptions. The effect of varying edge workload on the relative performance of different offloading strategies remains systematically understudied.

III. Methodology

A. System Model

We consider a standard two-tier MEC architecture consisting of mobile devices (MDs) and an edge server deployed at the base station. Computation offloading aims to migrate tasks from resource-constrained local devices to a more powerful edge server to improve latency and energy efficiency. The system operates in discrete time slots indexed by t, each with duration Δ.

At each time slot t generates a computation task characterized by its input data size L_t (in bits). The required number of CPU cycles is given by $C_t = cL_t$, where c denotes the computation intensity (cycles per bit). Task arrivals in the edge background follow a stochastic process, which can be approximated by a Poisson process with rate λ. Each task is associated with a latency constraint D.

1) Communication Model: We assume a block fading wireless channel, where the channel gain h_t remains constant within each time slot but evolves across slots according to a Markov process. The achievable uplink transmission rate is modeled as a finite-state Markov process with discrete rate levels, which can be interpreted as a quantized approximation of the Shannon capacity under different channel conditions. Accordingly, the transmission delay and energy consumption are

$$T_t^{\mathrm{tx}} = \frac{L_t}{R_t}, \quad E_t^{\mathrm{tx}} = p_t T_t^{\mathrm{tx}}. \tag{1}$$

We neglect the result downloading delay due to its relatively small size compared to the input data.

2) Computation Model: Each task can be either processed locally or offloaded to the edge server. To capture the temporal coupling introduced by unfinished workloads, we model both the mobile device and the edge server as maintaining computation backlogs measured in CPU cycles.

For local execution, let Q_t^{loc} denote the current local backlog at slot t, and let f^{loc} denote its CPU frequency. Then the local processing delay consists of the waiting time caused by the existing backlog and the execution time of the current task:

$$T_t^{\mathrm{loc}} = \frac{Q_t^{\mathrm{loc}}}{f^{\mathrm{loc}}} + \frac{C_t}{f^{\mathrm{loc}}}. \tag{2}$$

The corresponding local energy consumption is modeled under the standard DVFS model as $E_t^{\mathrm{loc}} = \kappa C_t (f^{\mathrm{loc}})^2$, where κ is the effective switched capacitance coefficient.

For edge execution, the total latency consists of transmission delay, edge waiting delay, and edge execution delay. Let Q_t^{edge} denote the current computation backlog at the edge server, measured in CPU cycles, and let f^{edge} denote the edge CPU frequency. Then the edge-side latency is given by

$$T_t^{\mathrm{edge}} = T_t^{\mathrm{tx}} + \frac{Q_t^{\mathrm{edge}}}{f^{\mathrm{edge}}} + \frac{C_t}{f^{\mathrm{edge}}}. \tag{3}$$

The local and edge backlogs evolve according to

$$Q_{t+1}^{\mathrm{loc}} = \left[Q_t^{\mathrm{loc}} + (1 - x_t) C_t - f^{\mathrm{loc}} \Delta \right]^+, \tag{4}$$

$$Q_{t+1}^{\mathrm{edge}} = \left[Q_t^{\mathrm{edge}} + x_t C_t + W_t - f^{\mathrm{edge}} \Delta \right]^+, \tag{5}$$

where $x_t \in \{0, 1\}$ is the offloading decision, with $x_t = 0$ indicating local execution and $x_t = 1$ indicating edge execution, and W_t denotes the exogenous background workload arriving at the edge server during slot t, measured in CPU cycles.

For compatibility with regime-based experiments, the external edge workload can be parameterized by a task-level arrival rate λ and an average service rate μ (in tasks per second), and the local device receives a task at the beginning of each time slot. In this case, the edge workload W_t is generated from a Poisson arrival process with rate λ, while the computing capabilities $f^{\mathrm{edge}}/f^{\mathrm{loc}}$ are set according to the average number of task cycles so that μ controls the effective edge service intensity.

3) Offloading Decision: We define a binary offloading decision variable $x_t \in \{0, 1\}$, where $x_t = 0$ indicates local execution and $x_t = 1$ indicates offloading to the edge. The resulting latency and energy consumption for each task can be expressed as

$$T_t = (1 - x_t) T_t^{\mathrm{loc}} + x_t T_t^{\mathrm{edge}}, \tag{6}$$

$$E_t = (1 - x_t) E_t^{\mathrm{loc}} + x_t E_t^{\mathrm{tx}}. \tag{7}$$

In this case, a task is regarded as failed if its end-to-end latency exceeds the deadline D.

B. MDP Formulation

We formulate the above sequential decision-making problem as a Markov Decision Process (MDP), denoted by $\langle \mathcal{S}, \mathcal{A}, P, r \rangle$. The system state $s_t \in \mathcal{S}$ captures the observable information, including channel condition g_t, edge queue length Q_t, device energy level b_t, and task features (L_t, C_t). The action $a_t \in \mathcal{A}$ corresponds to the offloading decision x_t.

The state transition probability $P(s_{t+1}|s_t, a_t)$ is determined by stochastic task arrivals, channel evolution, and queue dynamics. The reward function is defined as the negative system cost:

$$r_t = -\alpha_r T_{i,t} - \beta_r E_t - \gamma_{fail} \mathbf{1}(\text{failure}), \tag{8}$$

where a failure occurs when the latency constraint or energy constraint is violated.

The objective is to learn an optimal policy $\pi(a_t|s_t)$ that maximizes the expected discounted cumulative reward: $\max_\pi \ \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma_d^t r_t \right]$, where $\gamma_d \in (0, 1)$ is the discount

factor. This formulation enables the use of deep reinforcement learning methods, such as DQN or its variants, to learn efficient offloading policies under uncertainty.

C. State Representation and Temporal Modeling

Following the problem formulation in the previous section, we specify the state representation, history stacking, and stochastic transition process in this simulator.

In the beginning of time slot t, the local device observes its state information for the current state as well as states of the previous $n-1$ time steps, forming a history of length n.

$$\mathbf{H}^n(t) = \{s(t-n+1), ...s(t)\} \tag{9}$$

where the matrix $\mathbf{H}^n(t)$ denotes the history of all information in recent n states at time t, to facilitate the prediction of states in the near future. Each single-step state summarizes the observable system information:

$$s(t) = \big(g(t), Q^{\text{loc}}(t), Q^{\text{edge}}(t), b(t), L(t)\big) \tag{10}$$

where $g(t)$ denotes the wireless channel quality (discretized bandwidth level), $Q^{\text{loc}}(t), Q^{\text{edge}}(t)$ denote the local and edge backlogs, $b(t)$ is the remaining battery level, and $L(t)$ is the size of newly generated task at time step t, used also to calculate the CPU cycles. In each state s, each item is decided by the current time t based on its previous state, following the distribution discussed in the previous System Modeling.

The channel here is discretized into $K = 5$ channel states with an ordered bandwidth value. The channel evolves as a finite-state Markov chain following a channel transition matrix Tr. This Markov discretization provides temporally correlated channel variations while keeping the action space discrete.

Local device can obtain the state information $g_t, Q^{\text{loc}}(t), Q^{\text{edge}}(t), b(t), L(t)$ through its own state at the beginning of each time slot t. Then it chooses the action x for the task with action space $x \in \{0, 1\}$ for the current arrived task.

Let R be the reward defined as the negative system cost. Since we use the value-based RL (DQN family), the optimal action-value function satisfies the Bellman optimality equation:

$$Q_x^s = \sum_{s'} P(s' \mid s, x)\Big(R(s, x, s') + \gamma_d \max_{x'} Q_{x'}^{s'})\Big) \tag{11}$$

where $\gamma_d \in (0, 1)$ is the discount factor, and $P(s' \mid s, x)$ is the state transition probability.

Here, the state transition matrix $P(s' \mid s, x)$ is determined by three stochastic components and a set of deterministic updates:

- **Channel transition:** g_{t+1} follows the Markov transition matrix Tr.
- **Edge background workload:** additional exogenous workload arrives at the edge and perturbs Q^{edge}, modeled as Poisson arrivals in the implementation.
- **New task generation:** each time slot generates a new task, with input size L_{t+1} sampled from uniform distribution $L \sim Uniform(L_{min}, L_{max})$.
- **Deterministic state updates:** given realized channel, background, task samples, and the selected action, the battery and backlog terms can be updated by the execution, communication time, and the service processed within the slot.

This design makes s_t Markov, while the history window H_t^n provides a stronger representation for learning under temporal correlations like correlated states and edge workload fluctuations.

D. Reinforcement Learning Algorithms

Several RL variants are studied, differing in *(i)* Q-network architecture, *(ii)* replay strategy, and *(iii)* history window length.

1) MLP-DQN: MLP-DQN utilizes a normal Multi-Layer Perceptron (MLP) to perform the value function approximation: $Q_\theta(s_t, x_t) \approx Q^*(s_t, x_t), \quad x_t \in 0, 1$. This variant can use both the current state without history (i.e., $n = 1$) and historical information ($n = 10$) with and without PER, but all of them are processed by MLP networks. Here we use model dimension $n_{dim} = 64$ for the MLP approximator. This model serves as an ablation baseline for the DQN network in the experiment.

2) Attention-DQN: Attention-DQN replaces the MLP with a Transformer encoder to process the stacked history $\mathbf{H}_t^n \in \mathbb{R}^{n \times d}$. Each state in the window is projected to an embedding with positional encoding and passed through the encoder: $\mathbf{E}_t = \text{PE}\big(\mathbf{H}_t^n \mathbf{W}\text{input}\big), \mathbf{Z}_t = \text{Enc}(\mathbf{E}t), Q_\theta(\mathbf{H}^n t, \cdot) = f_\theta\big(\mathbf{Z}_t[n]\big)$, where $f_\theta(\cdot)$ is a small MLP head. Self-attention allows the agent to learn which past steps are more informative than others, resulting from perturbations such as persistent poor channel states or accumulating edge backlog, rather than treating all history steps equally. The encoder uses $n_{\text{heads}} = 4$ attention heads and feedforward hidden size $n_{\text{hidden}} = 128$.

The two architectures above define the base agents used in this study. To further investigate the impact of training and input design choices orthogonal to architecture, we additionally evaluate two enhancements that can be applied on top of either agent.

3) Prioritized Experience Replay: PER is not a standalone RL algorithm, but a training enhancement that improves sample efficiency by replaying high-error transitions more frequently. Each transition i is assigned priority $p_i = (|\delta_i| + \varepsilon)^\alpha$, yielding sampling probability $P(i) = p_i / \sum_k p_k$. To correct the resulting distribution shift, importance-sampling weights are applied [14]:

$$w_i = \frac{\big(N \cdot P(i)\big)^{-\beta}}{\max_j \big(N \cdot P(j)\big)^{-\beta}}, \tag{12}$$

with β annealed to 1 over training to progressively reduce bias. PER can be applied on top of any DQN variant and is evaluated here in combination with both MLP-DQN and Attention-DQN.

4) History Mechanism: The history mechanism is similarly an input representation choice rather than a separate algorithm — it can be paired with any architecture to extend its temporal receptive field. Two settings are compared: **no history** ($n = 1$, current state only) and **history** ($n = 10$), testing whether temporal context from previous states improves decision quality under correlated channel and workload dynamics. While MLP-DQN processes the flattened history as a single vector, Attention-DQN is specifically designed to exploit the sequential structure of the history window via self-attention.

IV. Experiment and Results

A. Experimental Setup

The simulator uses a binary action space (local execution vs. offloading), with setup summarized in Table I. Time is slotted with duration Δ, each task has a hard deadline D, and the wireless channel is discretized into $K = 5$ Markov states. At each slot, a task of size $L_t \sim \mathcal{U}(L_{\min}, L_{\max})$ requiring $C_t = cL_t$ CPU cycles is generated, while the edge server receives stochastic background workload modeled as Poisson arrivals.

Two RL agents, MLP-DQN and Attention-DQN, are evaluated with and without PER and history stacking. Baselines include **Always Local**, **Always Offload**, and a **Greedy Heuristic** that minimizes instantaneous delay. Under light load, this greedy policy is near-optimal since decisions are weakly coupled over time, while under heavier load, RL benefits from modeling long-term backlog dynamics.

TABLE I: Summary of Experimental Setup

Component	Setting
Action space	$\{0, 1\}$ (local vs. offload)
Channel model	$K = 5$ state Markov chain
Task model	$L_t \sim \mathcal{U}(L_{\min}, L_{\max})$, $C_t = cL_t$
Edge workload	Poisson background arrivals
RL models	MLP-DQN, Attention-DQN
Variants	PER, history ($n = 10$), no history ($n = 1$)
Baselines	Always Local, Always Offload, Greedy

We train value-based agents using ϵ-greedy exploration with replay buffers. Key parameters include discount factor $\gamma_d = 0.95$, learning rate with ϵ_r decaying from 1.0 to 0.005, replay capacity $N = 10^5$, and PER parameters $(\alpha, \beta) = (0.6, 0.4)$. Results are averaged over multiple episodes with fixed random seeds.

Performance is evaluated using latency, energy consumption, success rate, failure rate, and offloading ratio. The reward is defined as a weighted combination of latency and energy with a failure penalty, where $(\alpha_r, \beta_r, \gamma_{fail}) = (0.5, 0.3, 5.0)$, prioritizing task success. These metrics capture the latency–energy trade-off and robustness under varying workload conditions.

B. Results and Analysis

We explore two main experiments in this work: i) the effect of edge server workload on offloading strategy, and ii) the effect of RL algorithms, including model architecture, replay buffer, and history setting of each architecture.

1) Experiment I: Edge workload: We vary the system load by adjusting the ratio of the service rate μ to the arrival rate λ, and report results at different load levels (e.g., light, medium, busy, and overloaded). We choose the four levels of service rate $\mu = [4, 6, 8, 10]$ for overloaded, busy, medium, and light loads with $\lambda = 5$.

Under light load ($\mu = 2\lambda = 10$), the performance difference across always-offload, greedy, and DQN is small, as the system remains stable and the edge server is mostly available. In this regime, transfer latency is the dominant factor, and selective offloading suffices — both greedy and attention-DQN achieve $\sim$30% offloading rate while matching always-offload performance. The greedy heuristic achieves the highest overall performance, followed closely by DQN, since the optimal strategy under stable conditions can be well-approximated by minimizing instantaneous delay. Always-local performance remains constant across all experiments, as it does not depend on edge workload.

As load increases to the medium level ($\mu = 1.6\lambda = 8$), overall performance remains similar across greedy and DQN, but the strategy differs: greedy offloads $\sim$25% of tasks and DQN offloads $\sim$40%, almost twice as many tasks as the greedy algorithm. The performance of latency, energy, and success rate for greedy and DQN are all very close, with DQN slightly higher. While the difference is not big, the DQN algorithm starts to outperform greedy at this setting compared with the previous, possibly due to the greedy heuristic starting to be inefficient when the edge server is not always available. In this case, always-offload still yields reasonable performance as the edge server can handle most incoming tasks.

When the service rate approaches the arrival rate under busy load ($\mu = 1.2\lambda = 6$), the always-offload strategy degrades sharply — both trivial baselines drop to $\sim$10% success rate and 1×10^{-4} ms latency — while greedy and DQN maintain $\geq$80% success rate at $1/10\times$ the latency. The performance gap between greedy and DQN enlarged relative to medium load, and DQN's offloading rate decreased more than greedy, resulting in a similar offloading rate. Importantly, the performance gap from DQN to greedy is enlarged, since rising edge backlog makes instantaneous-only decisions suboptimal; DQN can learn to account for backlog accumulation and optimize decisions accordingly.

Under overload ($\mu = 0.8\lambda$), simple strategies degrade severely while RL-based methods remain comparatively stable. Always-offload latency nearly triples relative to the busy setting. The greedy heuristic degrades to as effective as the always-local behavior: since the edge backlog accumulates monotonically, offloading becomes a consistently risky decision, and greedy defaults to local execution in most cases. DQN, by contrast, learns to identify selective offloading opportunities from backlog patterns, achieving a $\sim$10% offloading rate compared to near-zero for greedy and always-local. This translates to 3000 ms latency, 0.56 mJ energy, and $>$90% success rate — significantly outperforming all baselines.

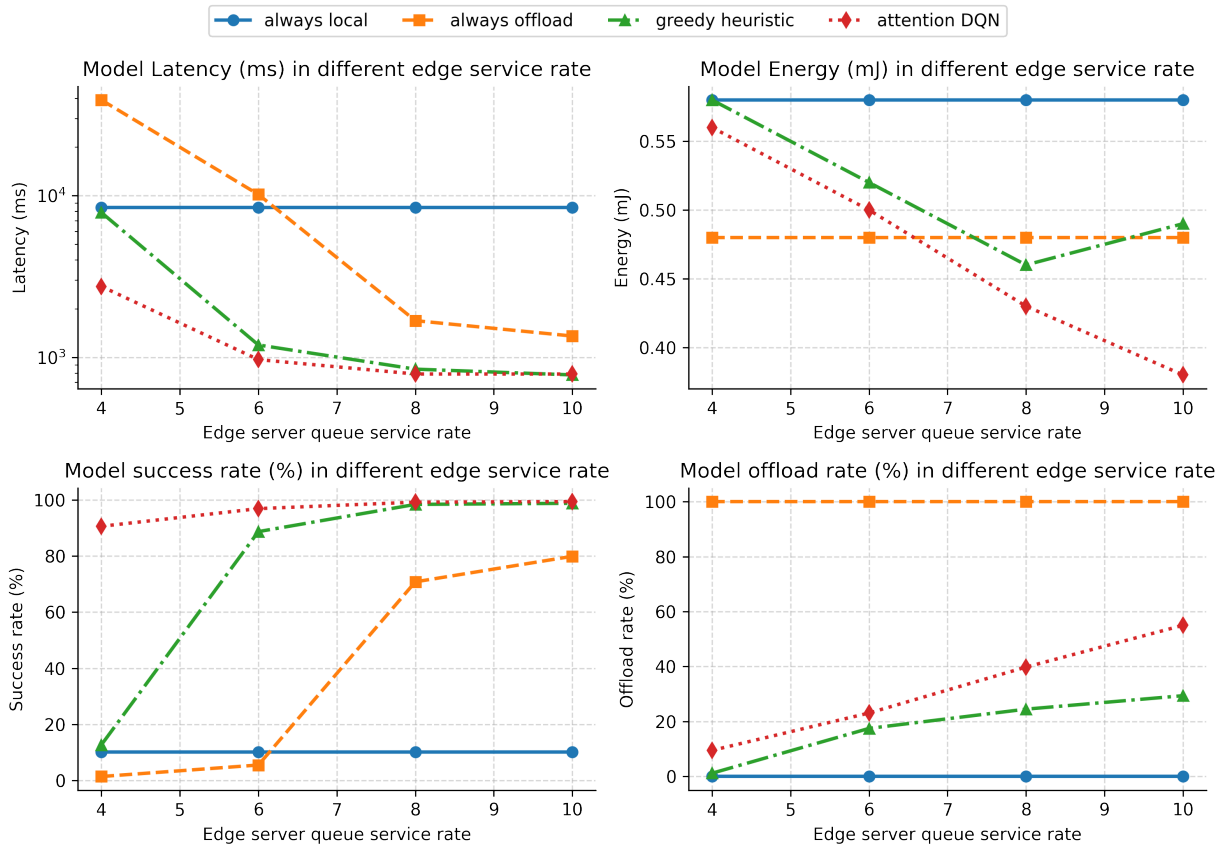

Fig. 1: Performance comparison of offloading strategies across edge service rates ($\mu = 4, 6, 8, 10$, with fixed $\lambda = 5$). Under light load, greedy and DQN achieve comparable low latency and high success rate ($\sim$99%) with selective offloading. As load increases, always-offload degrades sharply while RL-based methods remain stable. Under overload ($\mu = 4$), greedy collapses to always-local behavior, whereas attention-DQN maintains a higher offloading rate and superior performance across all metrics.

Overall, when edge load is low enough to prevent task accumulation, a greedy heuristic is sufficient, and RL provides limited added value. However, as load increases toward and beyond service capacity — a common scenario in real IoT deployments — simple strategies quickly fail while RL-based methods remain stable and close to optimal across all settings.

2) Experiment II: Representative design comparison: We further examine how different RL design choices affect performance, including architecture, replay strategy, and history usage. Six representative configurations are summarized in Table II, with optimal values highlighted in bold. Metrics include latency (ms), energy (mJ), and success rate (%).

Model architecture. The gap between MLP and attention models is consistently small. The best latency ranges from 743.6–765.7 (light) and 961.2–1019.1 (busy), with MLP+PER achieving the minimum in both light (743.6) and busy (961.2), while Attention+PER is best in medium (776.5). Energy differences are within $\sim$ 0.01–0.02 mJ across all settings (e.g., 0.379–0.415 in light). Success rates are similarly close: MLP peaks at 99.6%, while attention reaches 99.5%, with most differences $< 1\%$. This indicates that attention provides no consistent advantage over MLP in this setting.

Replay strategy (PER). PER yields minor and inconsistent improvements. MLP+PER achieves the best latency in 2/4 regimes and the lowest energy in medium (0.422) and overloaded (0.552). However, gains are small—for example, latency improves from 755.0 to 743.6 (light) and from 991.9 to 961.2 (busy), a reduction of $\sim 1\%$–3%. In contrast, the success rate slightly decreases (e.g., $99.6\% \rightarrow 99.5\%$). Overall, PER does not provide a clear or consistent benefit.

History mechanism. History also has a negligible impact. The highest success rate (99.6%) is achieved by both MLP+History and MLP+PER+History, but the margin over other methods is $< 0.1\%$. Latency varies within a narrow band (e.g., 755.0 vs. 743.6 in light), and energy differences are minimal ($<$ 0.01–0.02 mJ). No consistent improvement is observed across workloads, suggesting that the current state is already sufficiently informative.

Overall observation. The full model (Attention+PER+History) does not outperform simpler variants, with latency up to 765.7 (light) and 2743.4 (overloaded), both worse than several MLP-based settings. Across all metrics, performance variation remains within a narrow range: latency differences $< 5\%$, energy differences $< 5\%$,

TABLE II: Performance comparison of RL design configurations across workload regimes

Setting	Latency (ms)				Energy (mJ)				Success rate (%)			
	Light	Medium	Busy	Ove.	Light	Medium	Busy	Ove.	Light	Medium	Busy	Ove.
MLP+PER	**743.6**	789.2	**961.2**	2641.9	0.415	**0.422**	0.505	**0.552**	99.5	99.4	97.2	90.1
MLP+History	755.0	786.8	991.9	**2622.1**	0.393	0.427	0.512	0.557	**99.6**	**99.5**	**97.3**	90.3
MLP+PER+History	755.0	788.1	991.9	2630.3	0.393	0.426	0.512	0.555	**99.6**	**99.5**	**97.3**	90.2
Attention+PER	756.7	**776.5**	963.9	2754.3	0.388	0.434	0.501	0.556	99.4	99.2	97.1	**90.7**
Attention+History	**743.6**	782.0	1019.1	2772.9	**0.379**	0.440	0.501	0.558	99.5	99.1	96.7	90.5
Attention+PER+History	765.7	788.0	967.4	2743.4	0.380	0.425	**0.496**	0.559	99.4	99.2	96.9	90.6

and success rate differences $< 1\%$. These results indicate diminishing returns from added complexity, with simple designs such as MLP+PER or MLP+History achieving comparable performance at lower cost.

C. Limitation and Discussion

Our results show that the optimal offloading strategy strongly depends on edge workload. Under light load, simple heuristics can approximate optimal performance, making complex methods such as RL unnecessary. However, RL-based methods remain more stable across varying workload conditions, highlighting their advantage in generalizability. Within RL approaches, increased model complexity (e.g., attention mechanisms and PER) provides limited benefit and can even degrade performance. This is likely due to the relatively low-dimensional and near-Markovian nature of the problem, where most relevant information is captured by the current state. As a result, history-based methods offer minimal improvement. These findings suggest that RL design should remain aligned with problem complexity to avoid unnecessary overhead. Finally, the experimental setup is intentionally simplified to enable controlled analysis of workload effects. We consider a single-device, single-edge scenario with stochastic workload modeled via a Poisson process and channel dynamics captured by a Markov model. This design isolates the impact of edge load and algorithmic choices, allowing for clearer interpretation of results. While more complex multi-node or cloud-assisted systems may introduce additional dynamics, the current setup provides a clean and tractable benchmark for understanding fundamental offloading behavior.

V. Conclusion

This paper presented an empirical study of MEC offloading under varying edge workloads and RL design choices, targeting reusable AI for IoT edge deployment. Across four load regimes ($\lambda/\mu = 2.0, 1.6, 1.2, 0.8$), greedy heuristics suffice under light load ($\sim 99\%$ success rate) but collapse under overload, while RL remains stable above 90%, confirming RL's reusability advantage under variable, high-stress conditions. Within RL, attention, PER, and history stacking yield marginal gains within 5% latency and 1% success rate — simpler architectures like MLP+PER match or outperform the full model at lower cost, making them more viable for resource-constrained deployments. These findings support a practical design principle: favor lightweight RL components in near-Markovian MEC settings. Future work will extend to multi-device, multi-edge, and three-tier architectures, and explore policy transfer across heterogeneous IoT environments.

References

[1] T. Taleb, K. Samdanis, B. Mada, H. Flinck, S. Dutta, and D. Sabella, "On multi-access edge computing: A survey of the emerging 5g network edge cloud architecture and orchestration," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1657–1681, 2017.

[2] X. Xu, D. Li, Z. Dai, S. Li, and X. Chen, "A heuristic offloading method for deep learning edge services in 5g networks," *IEEE Access*, vol. 7, pp. 67734–67744, 2019.

[3] M. Guo, X. Huang, W. Wang, B. Liang, Y. Yang, L. Zhang, and L. Chen, "Hagp: A heuristic algorithm based on greedy policy for task offloading with reliability of mds in mec of the industrial internet," *Sensors*, vol. 21, no. 10, p. 3513, 2021.

[4] Z. Luo and X. Dai, "Reinforcement learning-based computation offloading in edge computing: Principles, methods, challenges," *Alexandria Engineering Journal*, vol. 108, pp. 89–107, 2024.

[5] M. Tang and V. W. Wong, "Deep reinforcement learning for task offloading in mobile edge computing systems," *IEEE transactions on mobile computing*, vol. 21, no. 6, pp. 1985–1997, 2020.

[6] A. M. Seid, G. O. Boateng, B. Mareri, G. Sun, and W. Jiang, "Multi-agent drl for task offloading and resource allocation in multi-uav enabled iot edge network," *IEEE Transactions on Network and Service Management*, vol. 18, no. 4, pp. 4531–4547, 2021.

[7] Y. Jia, C. Zhang, Y. Huang, and W. Zhang, "Lyapunov optimization based mobile edge computing for internet of vehicles systems," *IEEE Transactions on Communications*, vol. 70, no. 11, pp. 7418–7433, 2022.

[8] K. Tan, L. Feng, G. Dán, and M. Törngren, "Decentralized convex optimization for joint task offloading and resource allocation of vehicular edge computing systems," *IEEE Transactions on Vehicular Technology*, vol. 71, no. 12, pp. 13226–13241, 2022.

[9] Y. Tu, H. Chen, L. Yan, and X. Zhou, "Task offloading based on lstm prediction and deep reinforcement learning for efficient edge computing in iot," *Future Internet*, vol. 14, no. 2, p. 30, 2022.

[10] F. Khoramnejad and M. Erol-Kantarci, "On joint offloading and resource allocation: A double deep q-network approach," *IEEE Transactions on Cognitive Communications and Networking*, vol. 7, no. 4, pp. 1126–1141, 2021.

[11] W. Cheng, X. Liu, X. Wang, and G. Nie, "Task offloading and resource allocation for industrial internet of things: A double-dueling deep q-network approach," *IEEE Access*, vol. 10, pp. 103111–103120, 2022.

[12] X. Chen and G. Liu, "Federated deep reinforcement learning-based task offloading and resource allocation for smart cities in a mobile edge network," *Sensors*, vol. 22, no. 13, p. 4738, 2022.

[13] S. Moon and Y. Lim, "Federated deep reinforcement learning based task offloading with power control in vehicular edge computing," *Sensors*, vol. 22, no. 24, p. 9595, 2022.

[14] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," *arXiv preprint arXiv:1511.05952*, 2015.